Lancaster University Leipzig

Module handbook

Software Design Studio Project II (Networked Studio)

Academic Year: **2026/27**

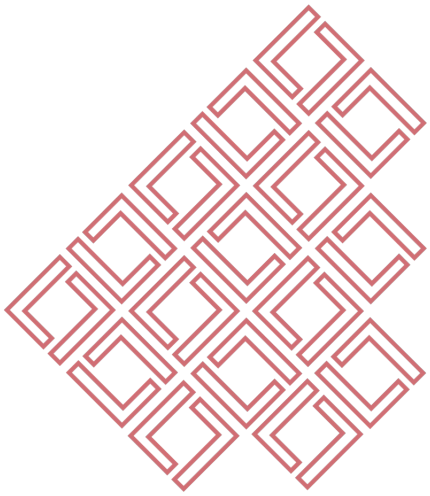
Credits: **30.0**

Module page URL: https://modules.lancaster.ac.uk/course/view.php?id=50277

Module Convenor: **Morteza Babazadeh Shareh**

Office hours: **On request**

Contact details: **m.babazadeh@lancaster.ac.uk**



1. Overview

LZSCC.230 gave students within small groups hands-on practical experience in the development of individual software modules. LZSCC.311 takes this learning a step further in introducing more complex and realistic software systems. Through the studio approach, students will focus on the integration and networking of software modules to create larger systems. In particular, students learn software engineering techniques relevant to medium scale networked projects such as models of distributed architecture, large-scale integration testing, distributed team development, and techniques for large scale software quality. As part of the project, teams will deliver reports, code, and demonstrate a working system. Furthermore, they will present certain elements of their work in written, graphical and verbal forms through the production of materials which may include reports, a website, poster and presentation.

2. Educational Aims

Subject Specific: Knowledge, Understanding and Skills

Provide students with knowledge in the application of software engineering, system development, and application programming principles and concepts to the development of networked software modules. Enable students to further develop their project management and planning skills alongside their technical skills (i.e. software and application design and programming) within a project and medium-size group context.

General: Knowledge, Understanding and Skills

Equip students with the ability to work within medium-size teams and as a team, to gain knowledge and experience on how to interact within a team, coordinate work within a group, and how to resolve problems and conflicts. Enable students to develop transferable skills in problem solving within the context of a reasonably sized (technical) process by applying knowledge and skills that have been acquired in

other modules throughout their studies. Provide students with skills in report writing, communication and presentation within a group context.

3. Learning Outcomes

Subject Specific: Knowledge, Understanding and Skills

At the end of the course the students should be able to work together in medium-size teams in order to tackle technical problems in the development of networked software modules, including analysing, planning, managing, designing, implementing, testing and fully documenting a system. They will have the knowledge and skills to be able to analyse a complex problem and design and develop a suitable solution. Students will develop new skills for coping with, and managing, parallel implementation. After the module, they will be able to produce documents to describe the design of a system and be able to critique those produced by others.

General: Knowledge, Understanding and Skills

Students will develop stronger interpersonal skills and a better understanding of cooperative work. They will learn group work techniques and how to apply these in different situations within a group. Further, they will also acquire additional verbal and graphical presentation skills, through the production of the outputs of the project.

4. Course Contents

In total, the module comprises 30 contact hours and 60 hours of independent learning. From week 1 to week 10, there will be 3 hours of guided lab time per week (Learning Sessions).

Week	Focus	Learning Activities
1	Foundation, Team Setup & Agile Fundamentals	Module introduction and team formation; Overview of networked systems; Scrum crash course (roles, ceremonies, artefacts); GitHub Issues and GitHub Projects for project management; Additional features selection (Tier

		1 + Tier 2)
2	Requirements & Microservices Architecture	Architecturally significant requirements; Microservices architectural style; Service decomposition strategies; API design and RESTful principles
3	Event-Driven Architecture & Docker Basics	Event-Driven Architecture (EDA) patterns; Message queues and event buses; Inter-service communication patterns; Event schema design; Introduction to Docker and containerisation
4	Integration & Implementation Sprint	System integration practices; Implementation support session; Sprint work time with technical guidance; Preparing for Assessment 1
5	Assessment 1 Presentations	Team presentations of first increment (core RAG pipeline); No new content; Feedback and reflection sessions
6	Scrum Refresher & Sprint 2 Planning	Sprint 1 retrospective; Sprint 2 planning for additional features; Scrum process review and adaptation; Team coordination for second increment
7	Advanced Testing & CI Foundations	Automated testing refresher (unit, integration, end-to-end tests); Introduction to Continuous Integration with GitHub Actions; CI pipeline concepts
8	CI Implementation & Code Quality	Hands-on GitHub Actions workshop; Code quality and review practices; Testing automation

		in CI
9	Integration & Final Implementation	System integration and testing; Additional features completion; Final implementation work; Acceptance criteria verification
10	Assessment 2 Presentations	Team presentations of complete system; No new content; Final demonstrations and reflection

5. Assessment

Assessment 1: First Increment — Core RAG Pipeline (50%)

Submission deadline: Before week 5 (upload to Moodle)

Presentation: Week 5

Focus: Functional core system with basic RAG capability, microservices architecture, event-driven design, and comprehensive documentation. This assessment represents the first 5-week sprint increment.

Group Component (50% of Assessment 1 = 25% of overall module grade): The team will demonstrate a working system including ingestion, indexing, retrieval, and basic RAG with at least one citation. The system must implement at least 3 events, run via Docker Compose, and include architecture and network documentation. Teams must also document the design of their chosen additional features (1× Tier 1 + 1× Tier 2) for implementation in Assessment 2.

Individual Component (50% of Assessment 1 = 25% of overall module grade): Each student will submit an individual reflection (2 pages maximum) on their technical contributions, architectural decisions, and the development process. The report should reference specific code contributions visible in the Git history.

Assessment 2: Second Increment — Production-Ready System (50%)

Submission deadline: Before week 10 (upload to Moodle)

Presentation: Week 10

Focus: Complete MVP with all acceptance criteria met, both additional features implemented, automated testing, and CI pipeline operational. This assessment represents the second 5-week sprint increment.

Group Component (50% of Assessment 2 = 25% of overall module grade): The team will demonstrate the complete system with full RAG capability (at least 2 citations), 5+ events, chat UI, both additional features working (1× Tier 1 + 1× Tier 2), automated tests, and GitHub Actions CI pipeline. The system must meet all MVP acceptance criteria including at least 90% correct citations in the demo catalogue. **Individual Component (50% of Assessment 2 = 25% of overall module grade):** Each student will submit an individual reflection (2 pages maximum) on their technical contributions to the second increment, testing approach, lessons learnt, and final system quality. The report should describe individual work on features, tests, or infrastructure visible in the Git history.

Team and Individual Assessment

Throughout weeks 2–10, each team must maintain their project artefacts via GitHub:

- Weekly updates to the repository (code, documentation, tests)
- Active use of GitHub Issues for task tracking
- Use of GitHub Projects for sprint planning and progress visualisation
- Regular commits from all team members

Students can demonstrate their individual contribution to the project through:

- Regular and meaningful Git commits with clear commit messages
- GitHub Issues created, commented on, and resolved
- Pull requests with code reviews and discussions
- Documentation contributions visible in the repository history
- Participation in GitHub Projects and sprint planning activities

6. Reading Material

The following books are recommended additional readings. You are not required to purchase these books, but they provide valuable background and deeper exploration of the module topics.

- **The Scrum Guide** by Ken Schwaber and Jeff Sutherland
    - Link: https://www.scrum.org/resources/scrum-guide
- **Microservices Patterns** by Chris Richardson
- **Practical Event-Driven Microservices Architecture** by Hugo Filipe Oliveira Rocha
- **Agile Software Development, Principles, Patterns, and Practices** by Robert C. Martin
- **Continuous Integration (CI) and Continuous Delivery (CD): A Practical Guide to Designing and Developing Pipelines** by Henry van Merode

7. Use of Generative AI Tools (RAG Classification)

The university's policies on artificial intelligence apply to all modules and can be found at: University Position on Artificial Intelligence.

This module has a **GREEN** RAG classification for the use of Generative AI tools. Students are **permitted and encouraged** to use Generative AI tools (such as GitHub Copilot, ChatGPT, Claude, etc.) to support their learning and project work, including:

- Ideation and brainstorming architectural solutions
- Drafting and refining documentation
- Code scaffolding and boilerplate generation
- Test case generation and testing strategies
- CI script creation and debugging
- Learning new technologies and troubleshooting errors

Requirements when using Gen AI tools:

- **Declare usage:** Document in your project README or assessment appendix which tools were used and for what purposes
- **Verify correctness:** All AI-generated code and content must be reviewed, tested, and validated by the team

- **Understand the code:** You must be able to explain and justify any code or design in your project, regardless of its origin
- **Security and privacy:** Do not input confidential data, personal information, or sensitive project details into public AI tools
- **Licensing:** Ensure AI-generated code complies with open-source licences and does not violate intellectual property
- **Academic integrity:** All work must be attributable to team members. Using AI tools does not exempt you from academic integrity requirements

8. Exceptional Circumstances

Exceptional Circumstances are unanticipated events that have had a significant impact on your ability to complete your assessments. Events associated with managing the day-to-day stresses of life, e.g., a common cold, assessment deadline proximity, normal exam nerves, minor sports injuries, difficulties with housemates/landlords, etc., will normally not be categorised as exceptional circumstances.

EC Request Procedure

- If you experience exceptional circumstances that impact your ability to complete an assessment, you may submit an EC request by filling in the EC form (available on the Moodle page)
- You need to submit the completed EC form along with **EVIDENCE** such as a doctor's note to Student Services at studentservices@lancasterleipzig.de
- **The above process must be completed within three (3) calendar days from the official deadline of the assessment**

The Exceptional Circumstances Committee will judge all EC requests at the end of the term. A more in-depth and detailed explanation can be found in Chapter "General Regulations for Assessment and Awards" Section 2.6 of the Manual of Academic Regulations and Procedures (MARP).